

## CHAPTER 7

# Architecture Repository



This chapter is part of the GREEN reading tour. See Figure 1-1 in Chapter 1, Section 1.3, for alternative reading tours.

---

*This chapter explores the concept of the Architecture Repository. The TOGAF Standard describes the repository as an intangible concept in which all aspects of the architecture are collected and stored. The chapter delves into the various aspects of the Architecture Repository and explains each one in detail. The relationship and connection between components are discussed at length, as is a method for making the repository tangible through appropriate architectural tools. One of the three continua described in this chapter can be used to access stored assets at any time. Key components that are stored in the repository, such as principles, building blocks, viewpoints and views, and the three continua are explained in more detail. How they can be created and used in the context of an architecture problem or challenge is covered in this chapter.*

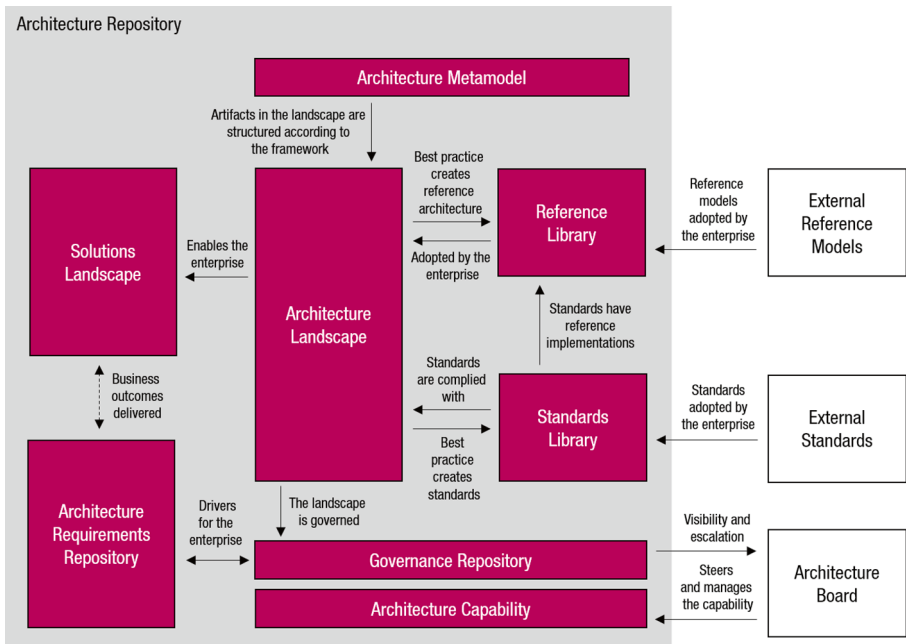
## 7.1. The Central Hub

Many organizations and their architects use architecture tools. These tools range from spreadsheets and slide decks to robust applications that provide nearly all the functionality of an Architecture Repository. Therefore, they are often considered to be equivalent to an Architecture Repository. More often than not, these tools contain numerous diagrams and other visualizations of solutions developed over time.

Yet, according to the TOGAF Standard, the Architecture Repository is an intangible concept. It is *not a tool* and should not be directly compared to one. Furthermore, an Architecture Repository should only contain reusable building blocks, an organization-specific metamodel, and additional architecture governance information. A well-designed repository *does not* contain diagrams of specific solutions.

The Architecture Repository acts as the backbone or central hub for the Enterprise Architecture practice. It's where all critical *architectural assets* – like frameworks, templates, and reference materials – come together. By offering a structured environment, it helps ensure that architectural development is smooth, efficient, and aligned with best practices. Whether this requires handling completed artifacts or works in progress, the Architecture Repository provides a clear organization for everything, streamlining management and making it easier to evolve the Enterprise Architecture as the needs of the organization grow.

The Architecture Repository (Figure 7-1) consists of a number of key components. Each of these components serves its own use. It also acts as an input to the perspective provided by the Enterprise, Architecture, and Solutions Continua.



**Figure 7-1.** *The Architecture Repository*

## 7.2. Key Components of the Architecture Repository

Reading the visualization of the Architecture Repository from top to bottom, the first item encountered is the *Architecture Metamodel*.

**Architecture Metamodel:** Defines the structure for how architecture artifacts are organized and how different types of information (like business, data, application, technology, and security) relate to each other.

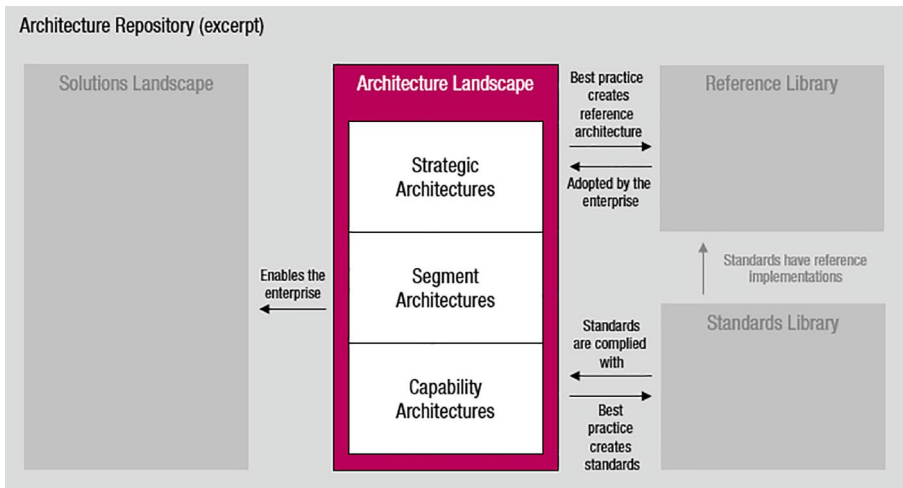
The Architecture Metamodel ensures consistency across architecture projects and helps architects to understand and relate different components of the architecture. It provides a common language, helping teams align across the organization. The Enterprise Metamodel (see

Chapter 5) serves as the foundation for the Architecture Metamodel and provides guidance for the *Architecture Landscape* to be created.

**Architecture Landscape:** This is a view of the organization's current, planned, and aspirational architecture. It typically includes different levels of architectures (strategic, segment, and capability) and timelines (past, present, and future).

Because the landscape can grow significantly over time and because of the different needs of stakeholders across the organization, the Architecture Landscape is divided into three levels of granularity (see Figure 7-2):

- **Strategic Architectures:** Provide a long-term summary view of the entire organization. They provide an organizing framework for operational and change activities and enable executive-level direction setting.
- **Segment Architectures:** Provide more detailed operating models for areas within an organization. Segment Architectures can be used at the program or portfolio level to organize and operationally align more detailed change activities. Another term often used for Segment Architectures is Domain Architectures. Figure 10-7 in Section 10.3.2 of Chapter 10 elaborates on Segment Architecture.
- **Capability Architectures:** Show in more detail how the organization can support a particular unit of capability. Capability Architectures are used to provide an overview of current capability, target capability, and capability increments and allow individual work packages and projects to be grouped into managed portfolios and programs.



**Figure 7-2.** Excerpt of the Architecture Repository, showing the three levels of architecture granularity

Strategic, Segment, and Capability Architectures play an important role in developing architecture using an agile approach. Chapter 18 details the use of agile architecture.

The Architecture Repository also provides a container for the Architecture Building Blocks associated with the Architecture Landscape. These reusable building blocks are created during the architecture work as a result of following the Architecture Development Method.

Furthermore, the Architecture Landscape enables the organization to do what it does by providing solutions through the *Solutions Landscape* in the form of reusable Solution Building Blocks. These building blocks are stored in the Solutions Landscape of the Architecture Repository.

**Solutions Landscape:** This contains models and documents related to individual Solution Architectures within the organization. It is typically more detailed than the broader Architecture Landscape allows for the creation of specific systems solutions or business solutions by using Solution Building Blocks.

The Solution Building Blocks stored in the Solutions Landscape support the Architecture Building Blocks that were specified, developed,

and deployed during the Architecture Development Method phases A, B, C, and D. The Solution Building Blocks can be products or services that can be categorized as Strategic, Segment, or Capability Solution Building Blocks according to the Architecture Building Block specifications.

---

It is important to note that the Solutions Landscape does not include the content (such as information and data) produced by the solutions selected within and by an organization; this is the responsibility of the solutions themselves [11].

---

The Solutions Landscape helps architects and project teams work on individual projects while keeping them aligned with the broader Enterprise Architecture. It is essential for solution delivery and integration efforts. Just below the Solutions Landscape is the *Architecture Requirements Repository*.

**Architecture Requirements Repository:** This repository captures, manages, and tracks architecture requirements, including both business and technical requirements.

The Architecture Requirements Repository helps ensure that architecture projects are aligned with the actual needs of the business. It also allows for the consistent management of requirements across different phases and projects, which is critical for ensuring that architectural outcomes meet organizational goals. All requirements created or updated during the course of the Architecture Development Method are captured and managed in this section of the Architecture Repository.

The architecture requirements associated with the Strategic, Segment, and Capability Architectures are stored in the Architecture Requirements Repository:

- **Strategic Architecture Requirements:** Offer a forward-looking perspective on the needs of an organization, outlining the necessary adjustments to support its

strategic direction. These requirements are pivotal in steering long-term transformations, ensuring that the organization aligns with its intended goals. Typically crafted at the executive level, they identify both the high-level direction and the specific shifts required to actualize it. Through this approach, Strategic Architecture Requirements serve as a roadmap for the organization's overarching vision and operational evolution.

- **Segment Architecture Requirements:** Offer precise, content-focused guidance for operational execution. Aligned with the organization's operating model, they break down broader strategic requirements into detailed, actionable steps. Typically crafted at the portfolio or project level, these requirements provide a more granular view, ensuring that necessary changes in architecture work are addressed comprehensively. In essence, Segment Architecture Requirements function as the tactical enablers that translate high-level goals into concrete, manageable actions within the architecture process. This level of detail ensures effective implementation and alignment with the organization's objectives.
- **Capability Architecture Requirements:** Focus on frameworks tied to specific capabilities or groups of capabilities. These requirements are crucial for defining the needs of individual initiatives, projects, or work packages within a broader project portfolio. The goal is to ensure that each piece aligns with the larger strategy, supporting the overall architecture while addressing the distinct objectives of each initiative.

The Architecture Requirements Repository is used during the execution of all phases of the Architecture Development Method. All requirements created or updated during the execution of the architecture process find their place in the Architecture Requirements Repository, where they are captured and managed.

To the right of the Architecture Landscape (for reference, see Figure 7-1) are two libraries situated: the *Reference Library* and the *Standards Library*. The first, the Reference Library, contains all kinds of reference material that can be used to develop architecture. This includes various bodies of knowledge, standards templates, and best practices.

**Reference Library:** This contains best practices, guidelines, templates, and patterns that can be reused in different architectural projects.

The Reference Library is a ready-to-use resource that accelerates the architecture development process. It includes reference architectures and models, templates, and the Viewpoint Library. The latter is a collection of views and viewpoints.

Architects can reference existing solutions, saving time and ensuring that the organization follows proven and standardized methods. This keeps solutions aligned and reusable across projects.

---

To make sense of the different classifications of reference materials, the TOGAF Standard refers to the Architecture Continuum (see Section 7.3.2). This continuum illustrates how reference architectures can be organized across a range – from Foundation and Common Systems Architectures to Industry and Organization-Specific Architectures [11].

---

The second library, as the name implies, contains standards in the form of specifications that the architecture must conform to.

**Standards Library:** A collection of standards and policies that guide the development and governance of the architecture. This includes IT standards, technology protocols, business process standards, and industry guidelines.



There are several types of standards, such as legal and regulatory mandates and industry and organizational standards. Both libraries (the Reference Library and the Standards Library) are fed by external reference models and standards. An example of an external reference model is the Business Architecture Guild Body of Knowledge, the BIZBOK Guide [5]. This guide serves as a reference for the practice and application of Business Architecture. Interestingly, the TOGAF Standard uses this body of knowledge as a reference. As for an external standard, the TOGAF Standard itself comes to mind. This open standard for architectural frameworks is called an external standard.

By providing architects and developers with a predefined set of standards, the Standards Library ensures compliance and consistency. It helps avoid redundancies and conflicts between different architecture components and projects.

The Architecture Landscape is governed by the Governance Repository and the Architecture Capability.

**Governance Repository:** A repository of decisions, rules, governance guidelines, and principles applied to architecture projects. It includes records of approvals, waivers, and compliance assessments.

The Architecture Landscape and Governance Repository provide decision-makers with the data and context needed for strategic planning, Risk Management, and operational alignment. Together with the Standards Library, the Governance Repository ensures that projects adhere to internal and external requirements, preventing non-compliance and ensuring regulatory alignment.

The Governance Repository thus provides accountability and traceability in architectural decisions, ensuring that all actions and developments adhere to the governance processes of the organization. This is critical for Risk Management and regulatory compliance.

**Architecture Capability:** This refers to the skills, processes, organizational structures, and roles that support the development and management of Enterprise Architecture.

The Architecture Capability helps in managing resources efficiently, ensuring that the organization has the right capabilities to deliver on architectural initiatives. It also supports continuous improvement in the Enterprise Architecture practice.

Both the Governance Repository and the Architecture Capability gain visibility through the Architecture Board. The Architecture Board is also used as an escalation mechanism. Conversely, from the perspective of the Architecture Board, it provides direction and management of the Architecture Capability.

The Architecture Repository is a powerful tool for managing architectural assets and supporting the governance, compliance, and collaboration necessary to deliver effective Enterprise Architecture. Its practical uses range from accelerating project delivery to ensuring consistency across all architecture initiatives within an organization.



#### Further Reading

Additional information about the Architecture Repository can be found in *The TOGAF Standard, 10th Edition*, document number C220.

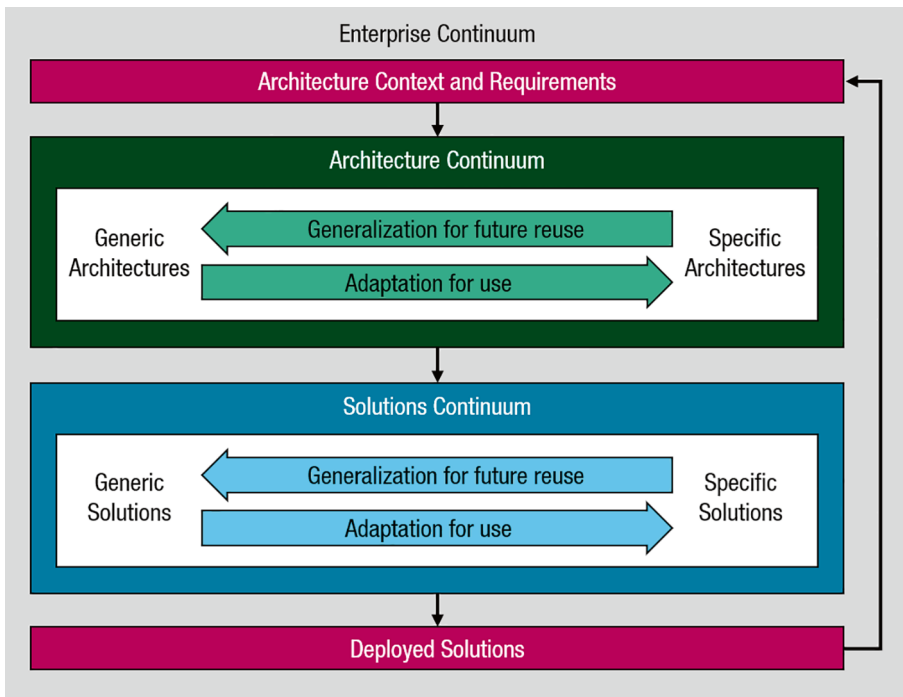
---

## 7.3. The Three Continua

As mentioned in the previous section, the TOGAF Standard refers to what is called a *continuum* to make sense of the different classifications of reference materials. It introduces the concept to enable a modular and adaptable approach to shaping an organization's architecture. A continuum allows architects to scale and customize development processes, adjusting the level of detail, complexity, and governance to fit specific needs. It's a versatile tool designed to accommodate the shifting requirements of various architectural landscapes, ensuring that organizations can evolve and structure their architectures effectively, regardless of the challenges they face. This aligns with the TOGAF Standard's overarching flexibility and adaptability as a framework.

A continuum in the context of the TOGAF Standard can be seen as *a lens through which to view the Architecture Repository*. A continuum is a *conceptual framework* that helps guide the creation and evolution of architectures and solutions.

The framework distinguishes three continua: the *Enterprise Continuum*, *Architecture Continuum*, and *Solutions Continuum* (see Figure 7-3). They allow the categorization of architecture assets stored and made available in the Architecture Repository. When applied from left to right, these three continua provide an increasingly detailed perspective (view) of the contents of the Architecture Repository.



**Figure 7-3.** *The Enterprise Continuum*

**Enterprise Continuum:** The Enterprise Continuum acts as an *umbrella* term that covers both the Architecture Continuum and the Solutions Continuum, bridging the gap between conceptual architectures (part of the Architecture Continuum) and practical solutions (part of the Solutions Continuum). The Enterprise Continuum provides a structure for understanding how assets are developed and reused across different levels, from the highly abstract to the practical. It helps an organization manage assets related to its architecture and solutions and allows the reuse of those architectural assets (such as building blocks).

**Architecture Continuum:** This continuum guides the creation and evolution of *architectures* by offering a range of options, from generalized, high-level architectures to more specific, concrete implementations. The Architecture Continuum allows architects to organize and classify different levels of architectural assets that an organization may need as it matures. It spans from very generic and abstract to specific and detailed.

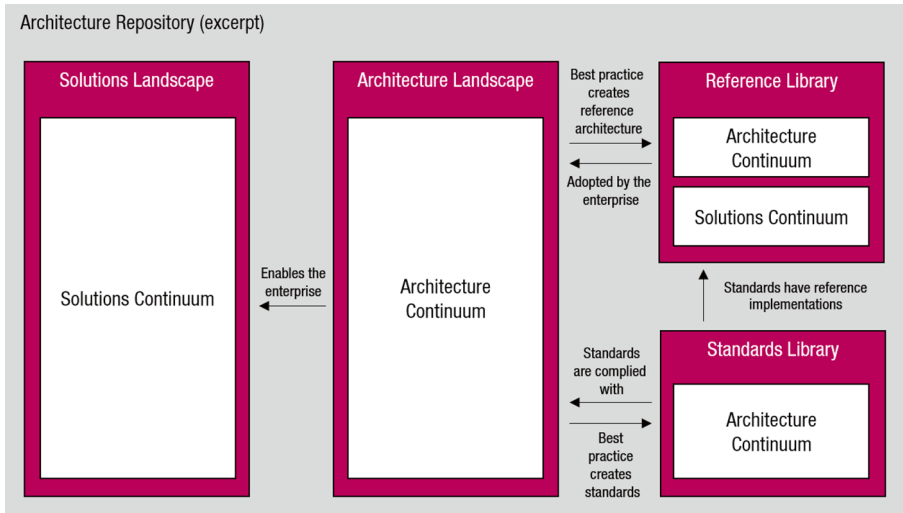
**Solutions Continuum:** The Solutions Continuum works in parallel with the Architecture Continuum but focuses on delivering actual, real-world *solutions and products*, aligned with the architecture. It shows how architectures are transformed into actual, deployable solutions, ranging from generic solutions to specific implementations.

Combined together, these three continua ensure a flexible approach to developing Enterprise Architecture, supporting scalability, reuse, and customization across various stages of organizational development.

The Architecture Repository acts as a centralized hub for all three continua, storing all architecture artifacts, standards, and references. It's the go-to resource for retrieving, managing versions, and applying governance controls, all in one place.

The setup demonstrated in Figure 7-4 streamlines the architecture effort, making it easier to maintain oversight. By organizing architectural content in a clear and structured manner, the Architecture Repository ensures that different teams and stakeholders stay on the same page.

Figure 7-4 highlights the locations of the Architecture Repository where the Architecture and Solutions Continua perspectives are concentrated.



**Figure 7-4.** Excerpt of the Architecture Repository, showing the views of the Architecture and Solutions Continua

### 7.3.1. The Enterprise Continuum

As with all three continua, the Enterprise Continuum is a conceptual framework and provides a lens through which to view the entire collection of architecture assets stored in the Architecture Repository. This can include everything from architecture descriptions and models to building blocks, patterns, and viewpoints. These assets are not limited to the organization, but also include best practices and standards from the broader industry.

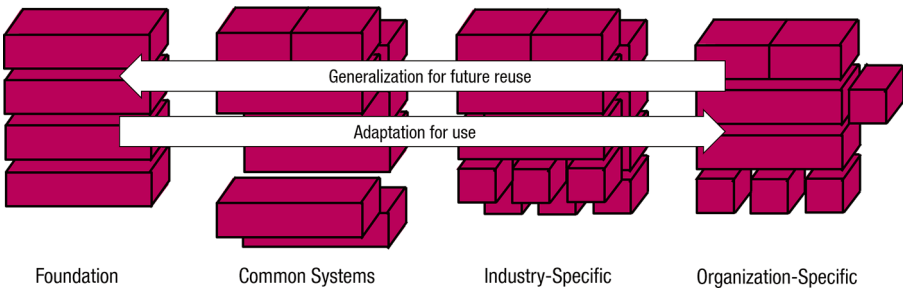
The Enterprise Continuum provides a view of the Architecture Repository that shows the evolution of these related architectures from generic to specific, from abstract to concrete, and from logical to physical [11].

---

The Enterprise Continuum is the *outermost continuum* and classifies assets related to the context of the overall Enterprise Architecture. It classifies contextual assets used to develop architectures, such as policies, standards, strategic initiatives, organizational structures, and organization-level capabilities.

Aside from the more high-level assets, the Enterprise Continuum can also classify solutions, as opposed to descriptions or specifications of solutions, which are provided by Solution Building Blocks. It contains two specializations, namely, the Architecture Continuum and Solutions Continuum.

The Enterprise Continuum provides a way to consider resources on a scale ranging from the most general (foundation) to most specific (organization-specific). This scale is visualized in Figure 7-5.



**Figure 7-5.** *The four classification models of the Enterprise Continuum*

Remember the Enterprise Metamodel described in Chapter 5?

This metamodel is a *Foundation* Enterprise Metamodel. This emphasizes yet again that the model should be used as a starting point and especially not as a single truth or a complete model that will suffice in every situation and for every organization.

The model expects (or even requires) modification to make it organization-specific.

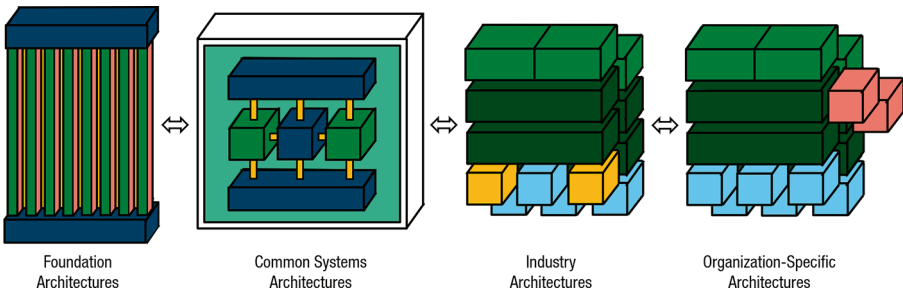
---

### 7.3.2. The Architecture Continuum

The structuring and categorization mechanism used for storing *architecture artifacts, standards, and reference architectures* in the Architecture Landscape, Reference Library, and Standards Library of the Architecture Repository is called the *Architecture Continuum*. This continuum guides the creation and evolution of architectures. It offers a range of options, from generalized, high-level architectures to more specific, concrete implementations.

The Architecture Continuum consists of four different architecture types that range from *foundational architectures* (generic) to *organization-specific architectures* (tailored for a particular organization). These architectures are used to define and deliver specific business capabilities, and the Architecture Continuum helps architects choose the appropriate architecture type based on the organization's needs.

The Architecture Continuum (Figure 7-6) illustrates how architectures are developed and evolved across the continuum ranging from Foundation Architectures through Common Systems Architectures and from Industry Architectures to an organization's own Organization-Specific Architecture.



**Figure 7-6.** *The four architecture types of the Architecture Continuum*

Figure 7-6 may be a little difficult to interpret. In particular, because parts of the Foundation Architectures and Common Systems Architectures models (the object shapes and sizes) are not clearly reflected in the models representing Industry Architectures and Organization-Specific Architectures.

It would be more evocative if, for example, parts of the Foundation and Common Systems Architectures are used in the model of, say, the Organization-Specific Architectures.

This has been done with the Industry Architectures model; parts of it can be found in the Organization-Specific Architectures model.

**Foundation Architectures:** A Foundation Architecture is a fundamental architecture upon which other, more specific architectures can be based. It consists of generic components, interrelationships, principles, and guidelines that provide a foundation on which more specific architectures can be built.

Foundation Architectures are the most generalized and abstract of the four architecture types. They provide a high-level reference model for universal services and technologies that can be used across various



industries. These architectures focus on fundamental building blocks such as network protocols, operating systems, security frameworks, and service-oriented architectures. Foundational components are crucial for ensuring that systems can operate efficiently and securely, but they are not specific to any specific industry or organization.

**Common Systems Architectures:** Common Systems Architectures build on top of the Foundation Architectures but are still generic across multiple industries. These architectures focus on more specific, common solutions that are applicable across different domains. They may include system-wide applications like content management systems, CRM, or data analytics platforms. Each is *incomplete* in terms of *overall system functionality*, but *complete* in terms of *a particular problem domain* (security, manageability, networking, operations, etc.).

For example, in healthcare, Common Systems Architectures might include general patient management systems, scheduling systems, or financial systems for billing and invoicing. These systems are necessary for running any healthcare organization, but they aren't necessarily unique to healthcare. They could be adapted from systems used in retail, education, or other industries. However, they must meet healthcare-specific regulatory requirements.

**Industry Architectures:** Industry Architectures are tailored to the needs of a specific industry. They provide reference models, guidelines, and standards that address the unique processes, requirements, and regulations of that particular industry. These architectures define best practices and common frameworks for industry-specific capabilities.

To illustrate, Industry Architectures would address the *unique workflows, standards, and regulations* relevant to the industry. To continue the example for the healthcare sector, this might include electronic health record (EHR) systems, telemedicine platforms, and health information

exchange (HIE) standards<sup>1</sup>. Industry-specific regulations like HL7 for data exchange<sup>2</sup> and ICD-11 coding for medical records<sup>3</sup> are key aspects of this architecture type. These architectures ensure that healthcare organizations can efficiently deliver patient care while meeting industry regulations.

**Organization-Specific Architectures:** Organization-Specific Architectures are the most tailored and customized of the four architecture types. These architectures are *designed specifically* for the unique needs of a *particular organization*, based on its goals, strategies, and environment. They reflect the specific workflows, business processes, and organizational culture.

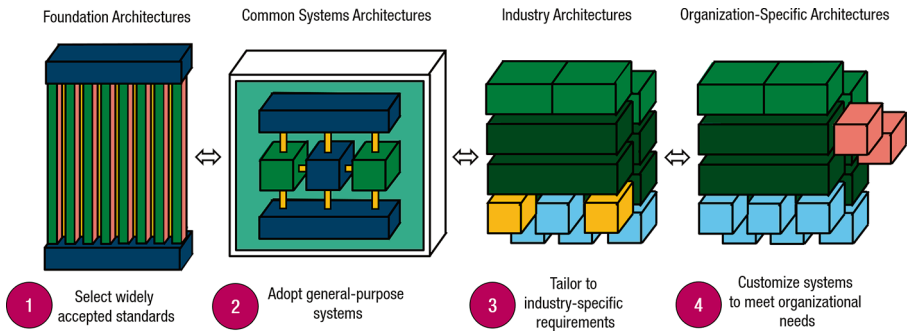
For a specific hospital or healthcare provider, the Organization-Specific Architecture might focus on implementing a customized EHR system that integrates seamlessly with the organization's unique workflows and patient management processes. It could involve bespoke features, such as a custom patient portal, integration with local pharmacy systems, and specific tools for managing population health. This type of architecture would also include specific configurations for mobile health applications, support for telemedicine, and integration with external systems, such as a local laboratory.

---

<sup>1</sup> A health information exchange (HIE) is a regional collaboration among independent healthcare organizations for sharing clinical information. Source: <https://www.gartner.com/en/information-technology/glossary/hie>

<sup>2</sup> Health Level Seven, abbreviated to HL7, is a range of global standards for the transfer of clinical and administrative health data between applications with the aim to improve patient outcomes and health system performance. Source: <https://www.hl7.org/>

<sup>3</sup> *International Classification of Diseases 11th Revision*. Source: <https://www.who.int/standards/classifications/classification-of-diseases>



**Figure 7-7.** Four steps for applying the four architecture types of the Architecture Continuum

As illustrated by Figure 7-7, the four types of architectures from the Architecture Continuum are typically applied in the following ways.

1. **Start with Foundation Architectures** to ensure that the components such as infrastructure and security are built on widely accepted standards.
2. **Move to Common Systems Architectures** to adopt general-purpose systems like billing or scheduling software.
3. **Use Industry Architectures** to tailor the systems according to industry-specific requirements. For healthcare, this could be patient records and regulatory compliance.
4. **Develop Organization-Specific Architectures** to customize those systems based on the specific needs of the organization.

A few years ago, I was working in the healthcare industry. I had the opportunity to work on the implementation of an Enterprise Architecture for several hospitals.

Many hospitals – at least in the Netherlands – are still in the early stages of working with architecture. The methodical approach and architectural thinking are still rather underdeveloped in this sector.

The challenge I faced at that time was to map the hospital across the axis of the things people do there. To put it in architectural terms, to map the business functions and the corresponding care processes.

Now, each hospital – depending on the specializations within the organization – fills out the processes and business functions in a slightly different way than the next hospital. This led me to look for a generic view of business functions and healthcare processes. I wanted something that I could use as a base and just add to or customize for the particular hospital I was working in.

That is how I ended up with ZiRA. This stands for the Dutch Hospital Reference Architecture.

ZiRA offered me a set of reference architectures that I could use as a foundation. For example, I was grateful to be able to use the existing business function model and the care process model. I was able to adapt both models to provide an accurate representation of the hospital I was working in.

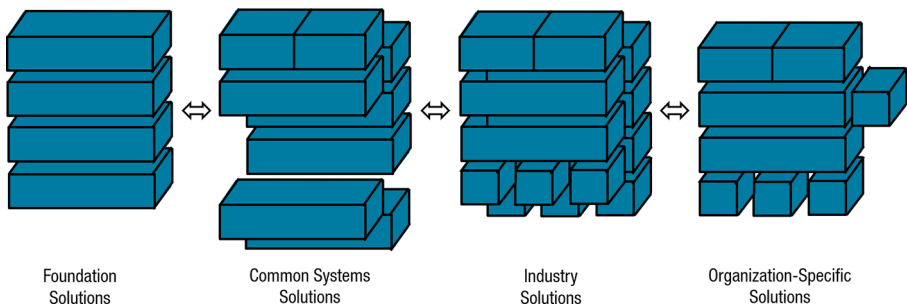
Reusing an existing reference architecture, in this case an Industry Architecture, helped me tremendously, not only in terms of design and interpretation, but especially in terms of time.

---

### 7.3.3. The Solutions Continuum

The Solutions Continuum is aimed specifically at artifacts related to solutions. Therefore, this continuum categorizes the components and artifacts stored in the Reference Library or Solutions Landscape with increasing detail and specialization (see Figure 7-4). It provides a consistent way to describe and understand the implementation of the assets defined in the Architecture Continuum.

The Solutions Continuum represents the detailed specification and construction of the architectures at the corresponding levels of the Architecture Continuum. At each level, the Solutions Continuum is a population of the architecture with reference building blocks – either purchased products or built components – that represent a solution to the organization's business need expressed at that level. A populated repository based on the Solutions Continuum can be regarded as a solutions inventory. Having such an inventory can add significant value to the task of managing and implementing improvements in the organization.



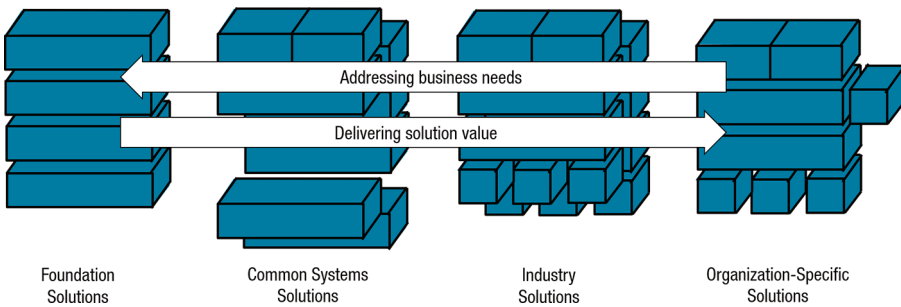
**Figure 7-8.** *The four solution types of the Solutions Continuum*

Moving from *left to right* on the Solutions Continuum (Figure 7-8) focuses on delivering solution value. For example, Foundation Solutions deliver value by creating Common Systems Solutions. These solutions are

then used to create Industry Solutions, and Industry Solutions are used to create Organization-Specific Solutions.

Moving from *right to left* on the Solutions Continuum focuses on addressing business needs.

These two perspectives are important to an organization that is trying to focus on its needs while maximizing the use of available resources through leverage. This is illustrated by Figure 7-9.



**Figure 7-9.** *Two perspectives of the Solution Continuum*

The Solutions Continuum contains the following four types of solutions:

**Foundation Solutions:** Highly generic concepts, tools, products, services, and solution components that are the fundamental providers of capabilities. Services include professional services, such as training and consulting services, that ensure the maximum investment value from solutions in the shortest possible time and support services, such as help desk, that ensure the maximum possible value from solutions (services that ensure timely updates and upgrades to the products and systems).

**Common Systems Solutions:** These are implementations of a Common Systems Architecture comprised of a set of products and services, which may be certified or branded. They represent the highest common denominator for one or more solutions in the industry segments that the Common Systems Solution supports.

Common Systems Solutions represent collections of common requirements and capabilities, rather than those specific to a particular customer or industry. They provide organizations with operating environments specific to operational and informational needs, such as high-availability transaction processing and scalable data warehousing systems. Examples of Common Systems Solutions include an enterprise management system product or a security system product.

**Industry Solutions:** Implementations of an Industry Architecture, which provides reusable packages of common components and services specific to an industry.

Fundamental components are provided by Common Systems Solutions and/or Foundation Solutions and are augmented with industry-specific components. Examples include a physical database schema or an industry-specific point-of-service device. Industry Solutions are – as the name implies – industry-specific and aggregate procurements that are ready to be tailored to an individual organization's requirements.

**Organization-Specific Solutions:** These solutions are the embodiment of implementations of the Organization-Specific Architecture that provides the required business capabilities. Because solutions are designed for specific business operations, they contain the highest amount of unique content in order to accommodate the varying people and processes of specific organizations.

Building Organization-Specific Solutions on Industry Solutions, Common Systems Solutions, and Foundation Solutions is the primary purpose of connecting the Architecture Continuum to the Solutions Continuum.



### Further Reading

More information about the Enterprise, Architecture, and Solutions Continua can be found in *The TOGAF Standard, 10th Edition*, document number C220.

---

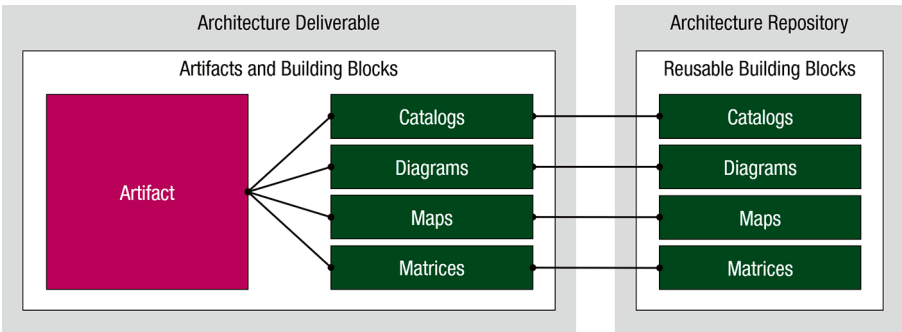
# 7.4. Catalogs, Diagrams, Maps, and Matrices

Many of the architecture assets or deliverables stored and categorized in the Architecture Repository consist of catalogs, diagrams, maps, and matrices.

Architecture deliverables are products created in the course of performing architecture work. They consist of one or more architecture artifacts, which are standardized architecture products for capturing and visualizing information related to the architecture work being performed.

Architecture artifacts, in turn, consist of one or more building blocks. These building blocks are documented using catalogs, diagrams, maps, and matrices and stored and categorized in the Architecture Repository.

The relationship of catalogs, diagrams, maps, and matrices to building blocks and architecture artifacts is shown in Figure 7-10. The figure shows that an architecture deliverable consists of one or more artifacts and that each artifact consists of one or more building blocks. A building block typically consists of one or more architecture entities. As mentioned earlier, the Architecture Repository provides storage for the building blocks.



**Figure 7-10.** Relationship between architecture deliverables, artifacts, and building blocks



Sections 7.4.1 through 7.4.3 clarify exactly what is meant by catalogs, diagrams, maps, and matrices. Examples are used to illustrate the differences and similarities.

### 7.4.1. Catalogs

Catalogs are *lists of building blocks* of a specific entity, type, or related types that are used for governance or reference purposes. A catalog addresses a specific entity (e.g., processes, applications, technology components) and supplements the information about the concept with additional data. In a catalog, entities are not linked to other entities. An example of a catalog is shown in Table 7-1.

**Table 7-1.** *Example of a Catalog*

| Entity Name | Value A | Value B |
|-------------|---------|---------|
| Entity A    | ...     | ...     |
| Entity B    | ...     | ...     |
| Entity C    | ...     | ...     |

A commonly used catalog is the Stakeholder Catalog (see Chapter 12, Section 12.4, Table 12-8, and Chapter 14, Section 14.3.2, Table 14-7). This catalog lists all stakeholders relevant to the architecture effort, along with their key concerns, classification, and interest (in the form of information provided).

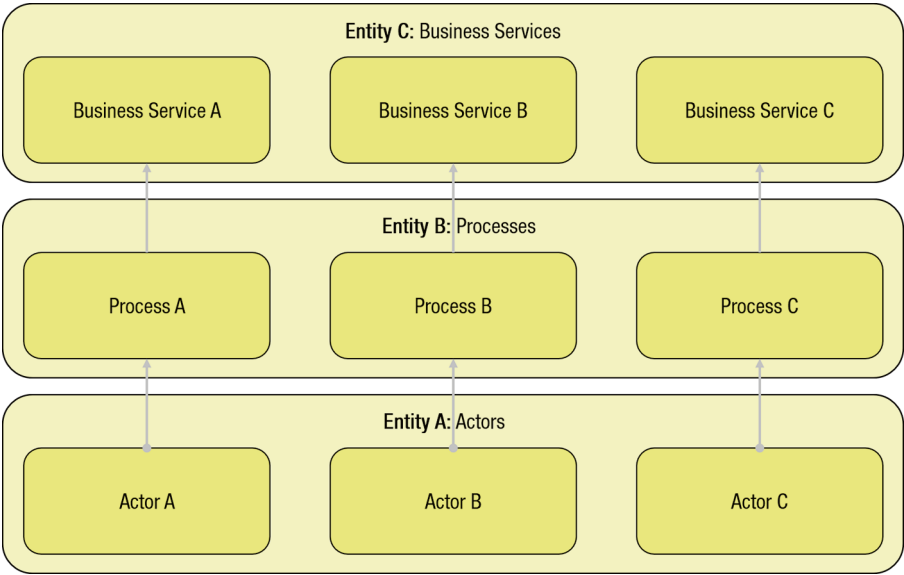
### 7.4.2. Diagrams and Maps

Diagrams and maps are *representations of architectural content* in either *graphical format* (diagrams) or *textual format* (maps). Diagrams and maps can also be used as a technique for graphically populating architectural content or for verifying the completeness of collected information (think,

e.g., of an organizational diagram in which locations and actors are added). A diagram or map visualizes information captured in a catalog or matrix.

**Table 7-2.** *Example of a Map*

| Entity A (Actor) | Entity B (Process) | Entity C (Business Service) |
|------------------|--------------------|-----------------------------|
| Actor A          | Process A          | Business Service A          |
| Actor B          | Process B          | Business Service B          |
| Actor C          | Process C          | Business Service C          |



**Figure 7-11.** *Example of a Diagram*

Figure 7-11 shows the same information as Table 7-2, but in a graphical form. Figure 7-11 is therefore a diagram, as opposed to Table 7-2, which is called a map.

One of the more known and often used diagrams is called the Process Flow Diagram. Such a diagram visualizes the steps necessary to perform a certain process in a graphical manner.

A well-known example of a map is the Business Capability Map or Organization Map (see Chapter 12, Section 12.4). The Business Capability Map lists the capabilities that an organization needs to fulfill its purposes. The capabilities may be supplemented by other entities such as processes and applications.

The Organization Map shows the relationships between the primary entities that make up the organization, its partners, and stakeholders.

7.4.3. Matrices

A matrix is a grid that represents the relationships between two or more model entities. The relationship between the entities – typically spread across two or more catalogs – is brought together in a matrix. The method of bringing entities together is called *cross-mapping*.

Table 7-3. Example of a Matrix

|                  | Entity B |          |          |
|------------------|----------|----------|----------|
|                  | Entity B | Entity B | Entity B |
| Entity A ▼       | Value A  | Value B  | Value C  |
| Entity A Value A | X        |          |          |
| Entity A Value B |          | X        |          |
| Entity A Value C |          |          | X        |

Table 7-3 shows two entities mapped together to create a cross-mapping. In the example, Entity A is cross-mapped to Entity B, to demonstrate their relationship. Entity A Value A has a direct relation with Entity B Value A, but not with Entity B Values B and C.

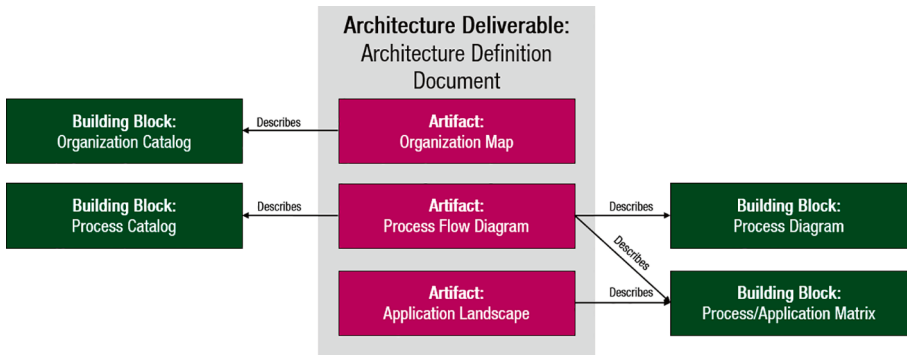
Knowing what to visualize – and what not to – is important for communicating with organizational stakeholders. As a general rule, less is more. Therefore, it is recommended to limit the amount of information used and displayed in a catalog, diagram, map, or matrix. It is better to create more views than to overload existing ones with information at the expense of the readability (and thus usability) of the architecture artifacts.

## 7.5. Architecture Deliverables

Architecture deliverables are an important part of the architecture being developed for an organization. They support the conversations that take place around architecture issues and provide insight into what certain situations or scenarios look like. Architecture deliverables are an essential link in the use of architecture artifacts to get the message across.

As mentioned earlier in Section 7.4, architecture deliverables are composed of one or more artifacts. Artifacts, in turn, consist of one or more building blocks. Building blocks include catalogs, diagrams, maps, and matrices.

The example in Figure 7-12 shows an architecture deliverable called the Architecture Definition Document. This deliverable consists of three architecture artifacts: an Organization Map, a Process Flow Diagram, and an Application Landscape. Each of the architecture artifacts consists of one or more building blocks. Architecture artifacts describe the (contents of) building blocks.



**Figure 7-12.** Example of an architecture deliverable and its contents

### 7.5.1. Architecture Artifacts

As one moves through the phases of the Architecture Development Method – whether through one of the iterative forms or not – a number of architecture artifacts may be produced.

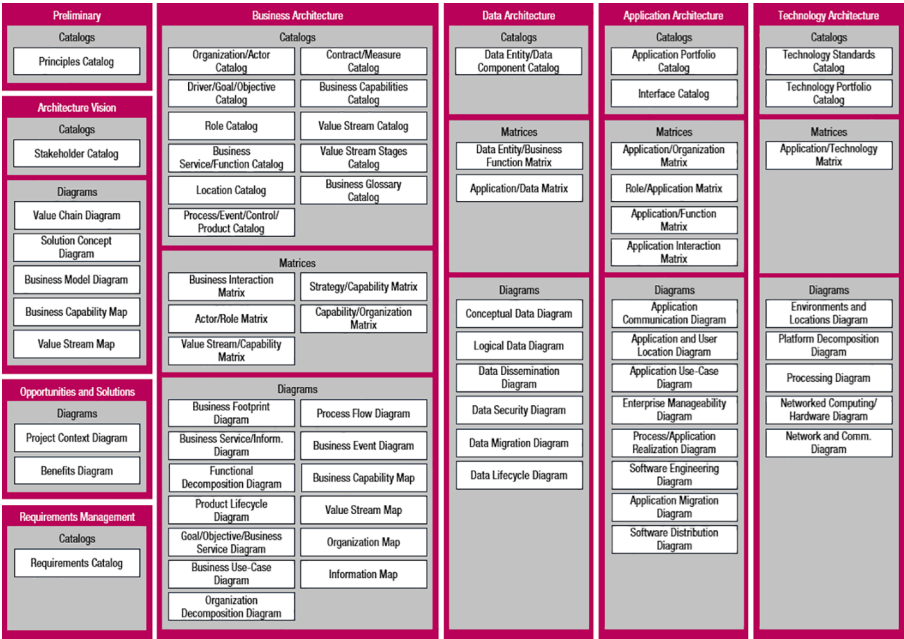
---

It's important to note that creating all the artifacts listed in the TOGAF Standard is by no means a necessity. The framework only indicates the possibilities, but in no way requires that all artifacts be created. Again, use those artifacts that make sense and are useful to the organization or architecture work.

---

Once an organization has adapted the Enterprise Metamodel to its specific situation or requirements, an overview of the necessary architecture artifacts is formed based on this model.

Figure 7-13 shows an overview of the possible architecture artifacts to be created based on the Foundation Enterprise Metamodel as provided in the TOGAF Standard<sup>4</sup>.



**Figure 7-13.** *Artifacts associated with the Foundation Enterprise Metamodel*

Architecture artifacts, as shown in Figure 7-13, are created during the Preliminary and Requirements Management phases and phases A-E of the Architecture Development Method (see Chapter 12, Sections 12.3 through 12.8 and Section 12.12).

<sup>4</sup> Figure 7-13 can be downloaded in a large format from <https://eawheel.com/books/media/>

As with all other components of the TOGAF Standard, Figure 7-13 and Table 7-4 show the possible artifacts that may be produced during the corresponding phases of the Architecture Development Method. Given the tailorability of the framework, it is certainly not necessary to produce all artifacts consistently.

It is very important to determine which artifacts actually add value to the architecture effort at hand and then create those artifacts. All other artifacts can then be ignored.

The recommended artifacts for production in each Architecture Development Method phase are as follows.

**Table 7-4.** *Artifacts and their entities created per ADM Phase*

| ADM Phase                       | Artifacts                | Used Entities                                                                                                     |
|---------------------------------|--------------------------|-------------------------------------------------------------------------------------------------------------------|
| Preliminary Phase               | Principles Catalog       | Principle                                                                                                         |
| Phase A:<br>Architecture Vision | Stakeholder Catalog      | – (or stakeholder, once added to the metamodel)                                                                   |
|                                 | Value Chain Diagram      | Value stream                                                                                                      |
|                                 | Solution Concept Diagram | Objective, requirement, and constraint                                                                            |
|                                 | Business Model Diagram   | Actor, role, stakeholder (see above), contract, function, process, business capability, business service, product |
|                                 | Business Capability Map  | Business capability                                                                                               |
|                                 | Value Stream Map         | Value stream                                                                                                      |

*(continued)*

**Table 7-4.** *(continued)*

| ADM Phase                      | Artifacts                             | Used Entities                                                                                        |
|--------------------------------|---------------------------------------|------------------------------------------------------------------------------------------------------|
| Phase B: Business Architecture | Organization/Actor Catalog            | Organization unit, actor, location (when Location Catalog is not maintained)                         |
|                                | Driver/Gol/Objective Catalog          | Organization unit, driver, goal, objective, measure (may optionally be included)                     |
|                                | Role Catalog                          | Role                                                                                                 |
|                                | Business Service/Function Catalog     | Organization unit, function, business service, application service (may optionally be included here) |
|                                | Location Catalog                      | Location                                                                                             |
|                                | Process/Event/Control/Product Catalog | Process, event, control, product                                                                     |
|                                | Contract/Measure Catalog              | Business service, application service (optionally), contract, measure                                |
|                                | Business Capabilities Catalog         | Business capability                                                                                  |
|                                | Value Stream Catalog                  | Value stream                                                                                         |
|                                | Value Stream Stages Catalog           | Business capability, value stream                                                                    |
|                                | Business Glossary Catalog             | –                                                                                                    |

*(continued)*



**Table 7-4.** *(continued)*

| <b>ADM Phase</b> | <b>Artifacts</b>                        | <b>Used Entities</b>                                                                                |
|------------------|-----------------------------------------|-----------------------------------------------------------------------------------------------------|
|                  | Business Interaction Matrix             | Organization unit, function, business service                                                       |
|                  | Actor/Role Matrix                       | Actor, role                                                                                         |
|                  | Value Stream/Capability Matrix          | Business capability, value stream                                                                   |
|                  | Strategy/Capability Matrix              | Business capability, value stream, course of action                                                 |
|                  | Capability/Organization Matrix          | Business capability, value stream, organization unit                                                |
|                  | Business Footprint Diagram              | Organization unit, function, business service                                                       |
|                  | Business Service/Information Diagram    | Business service, business information                                                              |
|                  | Functional Decomposition Diagram        | Capability                                                                                          |
|                  | Product Lifecycle Diagram               | Product                                                                                             |
|                  | Goal/Objective/Business Service Diagram | Goal, objective, business service                                                                   |
|                  | Business Use-Case Diagram               | Actor, role, process, function (later on, it can include data, application, and technology details) |
|                  | Organization Decomposition Diagram      | Actor, role, location                                                                               |

*(continued)*

**Table 7-4.** *(continued)*

| ADM Phase                  | Artifacts                            | Used Entities                                                                            |
|----------------------------|--------------------------------------|------------------------------------------------------------------------------------------|
| Phase C: Data Architecture | Process Flow Diagram                 | Process, all mappings related to the process entity                                      |
|                            | Business Event Diagram               | Event, process                                                                           |
|                            | Business Capability Map              | Business capability, entities that flesh out the business capability                     |
|                            | Value Stream Map                     | Business capability, value stream                                                        |
|                            | Organization Map                     | Organization unit, actor, role, stakeholder (if available)                               |
|                            | Information Map                      | Business information                                                                     |
|                            | Data Entity/Data Component Catalog   | Data entity, logical/physical data component                                             |
|                            | Data Entity/Business Function Matrix | Data entity, function (optionally, data entity relationship to owning organization unit) |
|                            | Application/Data Matrix              | Logical application component, data entity                                               |
|                            | Conceptual Data Diagram              | Data entity                                                                              |
|                            | Data Dissemination Diagram           | Data entity, business service, logical/physical application component                    |
|                            | Data Security Diagram                | Role, organization unit, logical/physical application component                          |
|                            | Data Migration Diagram               | Data entity, location                                                                    |
|                            | Data Lifecycle Diagram               | Data entity, process                                                                     |

*(continued)*

**Table 7-4.** *(continued)*

| <b>ADM Phase</b>                  | <b>Artifacts</b>                        | <b>Used Entities</b>                                                      |
|-----------------------------------|-----------------------------------------|---------------------------------------------------------------------------|
| Phase C: Application Architecture | Application Portfolio Catalog           | Application service, logical/physical application component               |
|                                   | Interface Catalog                       | Logical/physical application component                                    |
|                                   | Application/Organization Matrix         | Logical application component, organization unit, actor, business service |
|                                   | Role/Application Matrix                 | Logical application component, role                                       |
|                                   | Application/Function Matrix             | Logical application component, function, business service                 |
|                                   | Application Interaction Matrix          | Application service, logical/physical application component               |
|                                   | Application Communication Diagram       | Logical/physical application component, data entity                       |
|                                   | Application and User Location Diagram   | Logical/physical application component, location                          |
|                                   | Application Use-Case Diagram            | Actor, role, logical application component                                |
|                                   | Enterprise Manageability Diagram        | Logical/physical application component, logical technology component      |
|                                   | Process/Application Realization Diagram | Logical application component, process                                    |

*(continued)*

**Table 7-4.** *(continued)*

| ADM Phase                        | Artifacts                          | Used Entities                                                                                      |
|----------------------------------|------------------------------------|----------------------------------------------------------------------------------------------------|
| Phase D: Technology Architecture | Software Engineering Diagram       | Physical application component (composed of packages, modules, services, and operations)           |
|                                  | Application Migration Diagram      | Logical/physical application component, staging areas (plateaus), logical technology component     |
|                                  | Software Distribution Diagram      | Physical application component, physical technology component, location                            |
|                                  | Technology Standards Catalog       | Technology service, logical/physical technology component                                          |
|                                  | Technology Portfolio Catalog       | Technology service, logical/physical technology component                                          |
|                                  | Application/Technology Matrix      | Logical/physical application component, application service, logical/physical technology component |
|                                  | Environments and Locations Diagram | Logical/physical technology component, logical/physical application component, location, actor     |
|                                  | Platform Decomposition Diagram     | Logical/physical technology component                                                              |

*(continued)*

**Table 7-4.** *(continued)*

| ADM Phase                                  | Artifacts                                | Used Entities                                                                                                                                  |
|--------------------------------------------|------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Phase E:<br>Opportunities and<br>Solutions | Processing Diagram                       | Logical/physical technology component, logical/physical application component, actor, location                                                 |
|                                            | Networked Computing/<br>Hardware Diagram | Logical/physical application component, logical/physical technology component, function                                                        |
|                                            | Network and<br>Communications Diagram    | Logical/physical technology component                                                                                                          |
|                                            | Project Context Diagram                  | Work package, organization unit, function, business service, process, logical application component, data entity, logical technology component |
|                                            | Benefits Diagram                         | Stakeholder, once added to the metamodel, business capability, business service, product                                                       |

The various artifacts mentioned during the implementation of the phases of the Architecture Development Method listed in Table 7-4 are detailed in Chapter 12.

### 7.5.2. Building Blocks

The previous section covered the possible artifacts that can be created during phases A-E of the Architecture Development Method. It is important to recognize that each of these artifacts consists of one or more building blocks.

Building blocks are the backbone of the architecture and serve as essential, reusable components in architectural solutions. These blocks form a modular foundation, designed to be generic and adaptable, ensuring they can seamlessly integrate with other building blocks. Their inherent reusability enables architects to combine them in various configurations, promoting efficiency and scalability in developing architectural solutions. By focusing on reusability and modularity, building blocks not only streamline architecture development but also provide flexibility, allowing architects to tailor solutions that meet evolving needs while maintaining consistency.

---

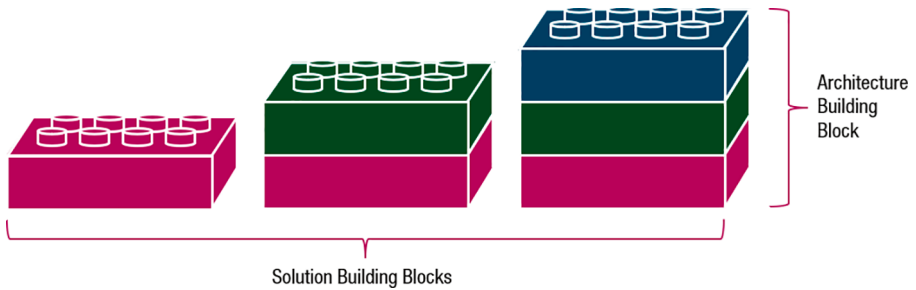
The comparison between building blocks and LEGO® is easy to make and also helps to understand how building blocks work.

The Danish toy company's bricks come in a variety of colors, shapes, and sizes. What they all have in common is that they are made so generically that they can easily be combined with other bricks to create new (sometimes larger) sets.

Building blocks in the context of the TOGAF Standard work much like LEGO bricks. They, too, can be stacked to create new building blocks.

For example, consider the Solution Building Blocks in Figure 7-14. The figure shows that stacking three Solution Building Blocks creates a larger one, an Architecture Building Block.

---



**Figure 7-14.** *Stackable LEGO bricks*

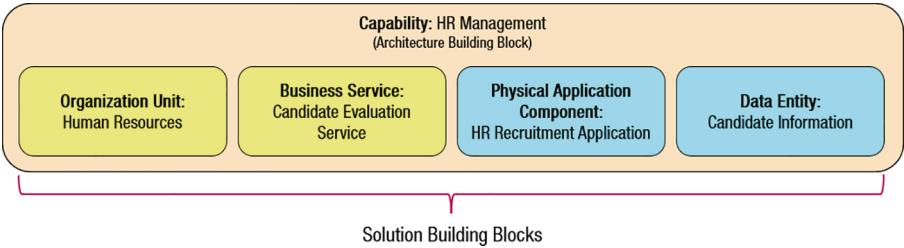
Flexible as they are, building blocks can be defined at varying levels of detail, depending on the stage of architecture development. In the early stages, a building block might just have a name or a brief description. As development progresses, it can be broken down into supporting components, complete with detailed specifications. Building blocks serve as key references for both *architecture* and *solution* design. This modular approach ensures adaptability across different phases and project requirements.

---

A building block represents a potentially reusable component that can be combined with other building blocks to deliver architectures and solutions [2].

---

Building blocks are packages of functionality defined to meet the business needs across an organization. They correspond to the entities used and defined in the Enterprise Metamodel (see Chapter 5), such as *organization unit*, *business service*, *application*, or *data entity*.



**Figure 7-15.** *Architecture Building Block composed of multiple Solution Building Blocks*

A building block is a well-defined unit with clear boundaries, designed to be useful and preferably reusable. Whether tangible or intangible, building blocks are meant to work seamlessly with other building blocks, enhancing their flexibility and adaptability in various architectural scenarios.

The TOGAF Enterprise Metamodel identifies the types of entity that need to be considered when generating a logical model, specifying a change (Architecture Building Blocks) and creating their associated physical and operational implementations (Solution Building Blocks) [16].

Because of the interaction with other components, it is imperative that the definition and scope of these building blocks should not be tightly bound to their implementation. In simpler terms, the building block's design should remain flexible enough to accommodate various implementations without altering its core structure. Different organizations will approach this assembly process uniquely, depending on their architecture.

When considering the level of detail for specifying a building block, it's crucial to align with the architecture's objectives. Sometimes, less detail can deliver more impact. For instance, when showcasing an organization's



capabilities using a Business Capabilities Catalog or a Business Capability Map, a single, clear diagram often holds more value than a lengthy, complex 100-page document. It's about clarity and precision – tailoring the information to what truly matters, ensuring that the message is concise and accessible, without overwhelming the audience with unnecessary depth.

### **7.5.2.1. Building Blocks in Architecture Design**

The various building blocks in an architecture specify the scope and approach that will be used to address a specific business problem.

---

An architecture is a set of building blocks depicted in an architectural model and a specification of how those building blocks are connected to meet the overall requirements of the business [11].

---

When designing specific architectures, certain core principles guide the use of building blocks:

- Only include building blocks that directly address the business problem at hand.
- Building blocks often interact in complex ways.
- A single building block may support multiple others or provide partial support to one.

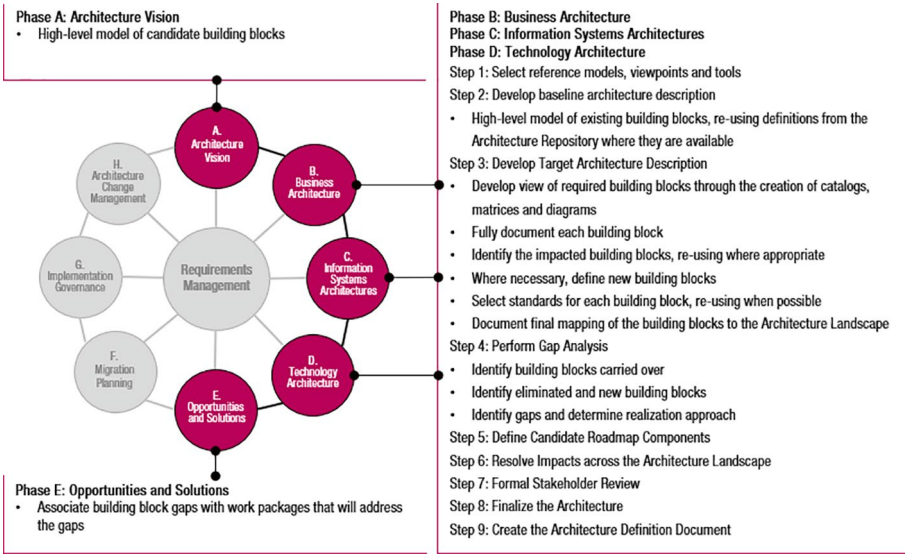
These principles ensure that the architecture remains relevant and appropriately structured to meet the unique challenges of the business.

In the early phases of developing architecture, building blocks tend to be more generalized. This broad approach helps ensure smooth integration across various components. But as projects move forward and the details of implementation come into play, these building blocks need to become more refined and precise. A clearer view of these elements

helps guide strategic decisions, highlights crucial choices, and evaluates the long-term impact of shared resources and reusability. By focusing on these critical aspects, the architecture becomes more aligned with both the strategic goals and operational demands of the organization. Each building block should adhere to the appropriate standards, Enterprise Principles, and any specific organizational guidelines.

7.5.2.2. Building Block Specification Process

The evolutionary and iterative process of building block definition takes place gradually as the Architecture Development Method is followed, mainly in phases A, B, C, and D. As the definition proceeds, detailed information about the functionality that is required, the constraints imposed on the architecture, and the availability of products may affect the choice and the content of building blocks.



**Figure 7-16.** ADM Phases and steps at which building blocks are evolved and specified

During Phase A (Architecture Vision), a high-level model of the candidate building blocks is created. The purpose of such a model is to identify the proposed change(s) required from an architectural perspective. This could include working out the required capabilities of an organization to implement the proposed change. An example of a high-level model typically created during Phase A (and updated and further specified in Phase B) is a Business Capability Map.

The execution of phases B, C, and D clarifies what building blocks are needed to further interpret the desired change(s). Phases B, C, and D specify the entities that flesh out the Business Capability Map created in Phase A.

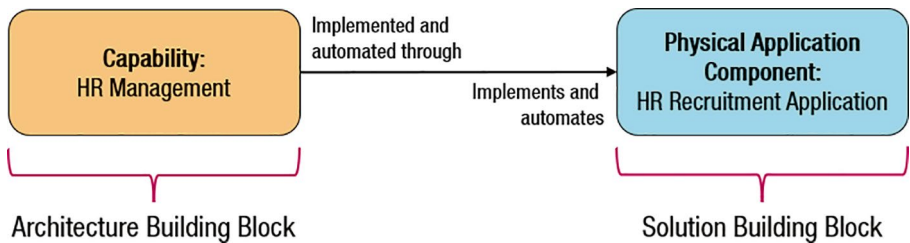
If the building blocks do not exist, they are created using catalogs, matrices, and diagrams. If they do exist, it is preferable to reuse them from the Architecture Repository. During phases A, B, and C, not only are new building blocks created or existing building blocks reused, but building blocks that are no longer needed are removed from the repository. This last activity ensures that the Architecture Repository is kept clean and up to date.

Building blocks are created during Phase E (Opportunities and Solutions) using work packages (initiatives). These initiatives fill in the gaps for the missing building blocks.

The key phases and steps of the Architecture Development Method at which building blocks are evolved and specified are summarized in Figure 7-16. The major work in these steps consists of identifying the Architecture Building Blocks required to meet the business goals and objectives. The selected set of Architecture Building Blocks is then refined in an iterative process to arrive at a set of Solution Building Blocks, which can either be bought off-the-shelf or custom developed.

### 7.5.2.3. Types of Building Blocks

Building blocks in the TOGAF Standard come in two flavors: *Architecture Building Blocks* and *Solution Building Blocks*. The distinction between them is simple but crucial. Architecture Building Blocks represent the *what* and *why*, providing high-level, technology-agnostic capabilities. Meanwhile, Solution Building Blocks dive into the *how*, focusing on practical implementation using specific technologies and products.



**Figure 7-17.** *Architecture Building Block capability implemented and automated through a Solution Building Block*

---

Note that Figure 7-17 shows an Architecture Building Block being implemented and automated by a Solution Building Block.

Normally, Solution Building Blocks only implement Architecture Building Blocks. However, in the case where the Architecture Building Block is a capability entity and the Solution Building Block is an entity from either the data, application, or technology domains, the Solution Building Block can also automate the Architecture Building Block [5].

---

By abstracting functionality and capabilities in Architecture Building Blocks and then implementing them through Solution Building Blocks (see Figure 7-17), the TOGAF Standard ensures the architecture remains flexible and scalable, while also grounded in practical solutions.

### 7.5.2.4. Architecture Building Blocks

When architecture work begins and the Architecture Development Methodology is applied during an initial iteration, Architecture Building Blocks are typically created during this round. These building blocks capture architecture requirements such as business, data, application, and technology requirements. The information stored in the building blocks from this activity can be used to direct and guide the development of Solution Building Blocks in a later phase of architecture development.

Architecture Building Blocks represent high-level conceptual components that define what needs to be delivered without specifying the technology or implementation details. They are concerned with requirements, capabilities, and strategic objectives, focusing on functionality, services, and constraints that are independent of specific technologies. Architecture Building Blocks typically help in shaping the architectural vision and form the blueprint for future solutions.

---

Architecture Building Blocks are architecture documentation and models from the enterprise's Architecture Repository [11].

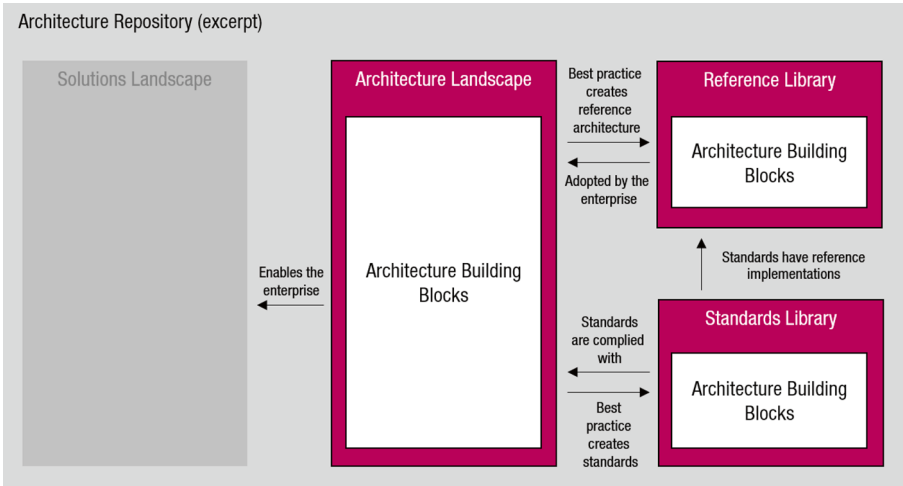
---

Specifications for Architecture Building Blocks include fundamental functionality and attributes, such as semantic and unambiguous attributes, security capabilities, and manageability. They also ensure interoperability and relationships with other building blocks. A final important detail of building block specifications is that they map to business or organizational entities.

Architecture Building Blocks typically describe a required capability and shape the specification of Solution Building Blocks. For example, an organization may require a Human Resources Management capability that is supported by a number of Solution Building Blocks, such as an organization unit, a business service, an application, and a data entity (see Figure 7-15).

The Architecture Building Block in this example outlines the need to manage HR-related activities, including, to name one, the acquisition of new employees. The capability, in this example, does not specify a particular product, process, or service, so it remains high-level.

Although not explicitly named in the TOGAF Standard, Architecture Building Blocks are typically stored in the Architecture Repository. Since this repository serves as a central location for all architectural assets, it includes both current and reusable assets.



**Figure 7-18.** Excerpt of the Architecture Repository, showing Architecture Building Blocks storage locations

Within the Architecture Repository, Architecture Building Blocks are stored in the following sections (see Figure 7-18):

- **Architecture Landscape:** This stores various architectural models at different levels of abstraction, including the Architecture Building Blocks that define architecture elements at different stages of architecture development.

- **Reference Library:** This section contains reusable architecture assets like architecture patterns, principles, standards, and Architecture Building Blocks that can be referenced when developing architectures.
- **Standards Library:** Architecture Building Blocks can be stored here when they align with industry standards, helping to ensure consistency with established guidelines.

These sections within the Architecture Repository allow organizations to store, manage, and govern Architecture Building Blocks effectively as part of their architecture practice.

### 7.5.2.5. Solution Building Blocks

In contrast to the high-level character of Architecture Building Blocks, Solution Building Blocks are actually much more detailed in nature. In fact, these building blocks describe the entities that make up the parent Architecture Building Block. Solution Building Blocks define what products and components will implement the required functionality; they determine the actual implementation of (parts of) the architecture. Solution Building Blocks fulfil business requirements and are product- or vendor-aware, something that Architecture Building Blocks are not.

---

Solution Building Blocks represent actual implementation choices to realize a required capability [2].

---

For example, a business service such as the Candidate Evaluation Service mentioned in Figure 7-15 is a Solution Building Block, since it can be described through complementary artifacts and then put to use to realize solutions for the organization. Solution Building Blocks may be either procured or developed.

Specifications for Solution Building Blocks include specific functionality and attributes. These building blocks map to the IT topology and operational policies. The attributes of Solution Building Blocks are aimed at, for example, security, manageability, localizability, and scalability, but also at performance and configurability.

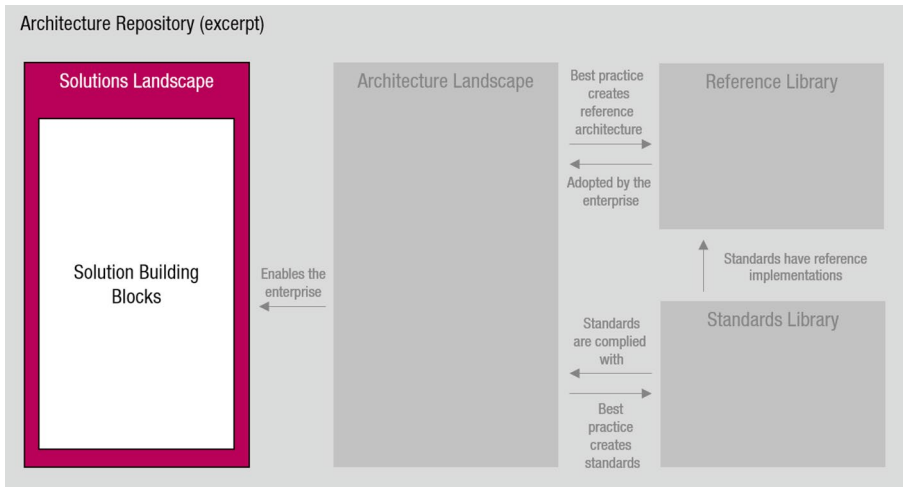
One final specification of Solution Building Blocks is their relationship to Architecture Building Blocks and other Solution Building Blocks.

Solution Building Blocks represent physical or concrete components that implement the functionalities or capabilities defined by the Architecture Building Blocks. They include specific technologies, products, and solutions that are deployed to realize the architecture. Solution Building Blocks are directly linked to real-world implementation and include the detailed design of systems, platforms, and technologies that meet the requirements laid out by Architecture Building Blocks.

The aforementioned HR Management capability (Architecture Building Block) did not provide specific details as to which application could support the capability. This is where the Solution Building Block comes into play. This building block defines the actual application (by using the logical or physical application component entity) that automates the HR Management capability. The specific product provides all the required functionalities.

Just as the TOGAF Standard does not mention where the Architecture Building Blocks are stored, it also omits a description of the location of the Solution Building Blocks.





**Figure 7-19.** Excerpt of the Architecture Repository, showing the Solution Building Blocks storage location

Solution Building Blocks are stored in the Architecture Repository's following section (see Figure 7-19):

- **Solutions Landscape:** This stores the Solution Building Blocks at different levels of abstraction. These can be shaped as catalogs, diagrams and maps, or matrices.



### Further Reading

Additional information on building blocks can be found in *The TOGAF Standard, 10th Edition*, document number G220, and the TOGAF Series Guide "An Approach to Selecting Building Blocks", document number G248.

## 7.6. Views and Viewpoints

To get the message of the architecture and its work right, it is important to understand the concerns of the stakeholders involved. This is a prerequisite for communicating the final architecture from the right perspective. To accomplish this, the TOGAF Standard refers to the use of views and viewpoints. The two concepts are similar, yet fundamentally different. An understanding of both concepts is necessary in order to convey the architecture's message in a sound manner. This section explores the differences and similarities between the concepts.

In the TOGAF Standard, the terms *view* and *viewpoint* are key concepts used in architectural descriptions. It is important to have a clear understanding of what they are and how they differ from each other.

**View:** A representation of a system or architecture from the *perspective of a specific stakeholder*. It is essentially a *visualization* or *presentation* of the architecture that addresses particular concerns or interests of one or more stakeholders.

The purpose of a view is to help communicate architectural decisions or system properties to stakeholders in a way that is meaningful to them. For example, a security view might describe the security measures in place for a system, while a process view could show how business processes flow through the architecture.

**Viewpoint:** Defines the *perspective* and *rules* for creating a view. It is essentially a *template* or a *set of guidelines* for constructing views to address specific concerns. Viewpoints outline *how* to structure views and which stakeholders they are aimed at.

It ensures that views are created in a consistent manner and address relevant concerns of stakeholders. A performance viewpoint would specify how to create views that deal with performance-related aspects of the system, like response times or throughput.

## 7.6.1. Relationship Between Views and Viewpoints

A viewpoint is like a *blueprint* for constructing views. Multiple views can be created using a single viewpoint, but each view is tailored to specific stakeholder concerns.

In summary, a *view* is what you actually present or show to stakeholders, while a *viewpoint* is the set of rules or guidelines that dictate how you create that view.



**Figure 7-20.** *Two viewpoints and one view*

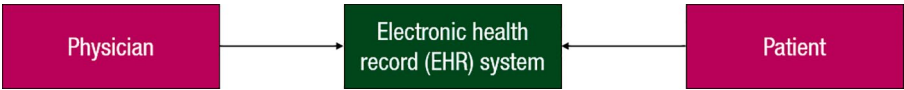
Views and viewpoints are part of the Reference Library of the Architecture Repository. This part of the repository contains organizational reference material, which includes views and viewpoints. The differences and similarities between the two concepts are best illustrated by a simple example (as depicted in Figure 7-20).

Imagine the following situation: There is a small village located in a valley. This valley is surrounded by two mountain peaks, one to the left and one to the right of the village. On each mountaintop there is a pair of binoculars used to look at the village. Each pair of binoculars is used by a different person, one on each mountaintop. Both people are looking through the binoculars at the village in the valley. The person on

the left mountaintop may be focusing on the church tower in the village, while the person on the right mountaintop is focusing on the green park surrounding the village.

Both people are looking at the same village, but may see it slightly differently. Perhaps this is due to the position of the binoculars, so that the person who focuses on the church tower cannot see the green square at all. Another possibility is that this person is not interested in the green square at all and therefore focuses on the church tower. Two people looking at the same village from two different vantage points.

In this example, the individuals are the stakeholders within an organization, and the binoculars used on the two mountaintops are the viewpoints from which the village is viewed. In this example, the village represents the view.



**Figure 7-21.** *Two viewpoints and one view in a real-world scenario*

Let’s consider a real-world scenario as shown in Figure 7-21. A physician and a patient both *looking* at the same system, in this case the electronic health record (EHR) system. The physician views the EHR system as a tool to efficiently manage patient information, support (real-time) decision-making, and streamline administrative tasks. The patient, on the other hand, views the EHR system as a tool that focuses on accessing their medical information, communicating with healthcare providers, and managing their health in a convenient and user-friendly format.

In practice, different stakeholders may view the same system, situation, or scenario differently. This may be due to constraints, specific requirements, or specific expectations or concerns of the stakeholders. Therefore, it is very important to create multiple viewpoints for the same view.



### Further Reading

For more information about views and viewpoints, consult *The TOGAF Standard, 10th Edition*, document number C220.

## 7.7. Using the Architecture Repository

The Architecture Repository is - as are most concepts described in the TOGAF Standard - positioned as an intangible object (or concept). This can make it difficult to translate the concept of the repository into an implementation suitable for everyday practice. Fortunately, the repository can be made more tangible by introducing and acquiring an architecture tool. Most architecture tools offer excellent ways to document and store essential architectural artifacts such as catalogs, diagrams, maps, and matrices. Architecture and Solution Building Blocks (the latter being responsible for providing content to the catalogs, diagrams, maps, and matrices) can also often be stored inside an architecture tool.

There are many tools on the market today, and each one offers advantages over the others. It is therefore important to think carefully about what is required of such an architecture tool (see Table 7-5).

In a small organization, it may initially seem possible to use a word processor or presentation software (or even a spreadsheet application) as a tool for maintaining the Architecture Repository. However, it will soon become apparent that these (in themselves excellent) products do not provide the capabilities that an architect would want or need. Besides the fact that these types of tools do not support a unified and globally accepted modeling language, it also means that each architecture artifact is created independently of other artifacts. This implies that each artifact must be manually tracked. As an organization and its architecture grow and take on increasingly voluminous forms, this becomes impossible to keep up with.

Tools that provide some sort of database construction already come close to what is needed. One of the most important things an architecture tool should provide is the ability to relate the entities used within the context of the architecture. By being able to properly establish and maintain these relationships, changes can be made quickly and accurately, without loss of context and without having to do it all over again for each individual artifact.

### **7.7.1. Deciding on a Tool**

It was mentioned at the beginning of this chapter that the Architecture Repository is an intangible concept and is not necessarily equivalent to an architecture tool. However, many organizations use a tool under the assumption that it is.

Vendors of architecture tools offer a variety of features in their products that come close to meeting the requirements of an Architecture Repository. The use of an architecture tool is therefore essential.

Deciding which tool is best for the organization depends on several factors. Budget, for example, is one such important factor. If sufficient budget is available for the purchase of a solid architecture tool, the organization can choose a product that emerges as a market leader. By purchasing such a product, the organization will have access to all possible functionality, and the creation and management of architectural artifacts will become much easier. If there is less budget available, the organization will be forced to purchase a tool that automatically offers less functionality.

Every year, Gartner produces an overview and report of the market leaders in the field of architecture tools [17]. This report can be used to select a suitable product. The availability of an architecture tool is an absolute must in order to turn the Architecture Repository described by the TOGAF Standard into the aforementioned tangible solution.

Some considerations that can be used in the search for an appropriate tool are shown in Table 7-5. These considerations are in addition to the fact that the tool must have at least solid repository functionality.

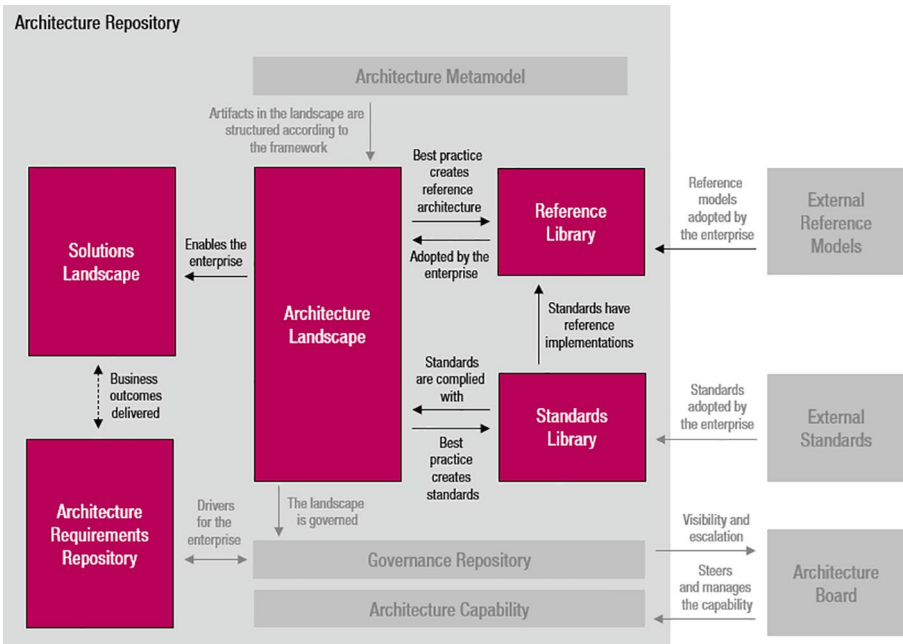
**Table 7-5.** *Architecture tool functionality considerations*

| Area of Functionality           | Considerations                                                                                                                                                                                                                                                   |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Support for a modeling language | <p>Does the tool support the use of a modeling language (such as ArchiMate) in its entirety?</p> <p>Can user-defined properties be added to elements and relationships?</p> <p>Is the use of BPMN supported?</p> <p>Is it possible to create colored views?</p>  |
| Metamodel customization         | <p>Does the tool allow adjustments to its metamodel?</p> <p>Can the tool be adapted to personal or organizational needs?</p>                                                                                                                                     |
| Tool flexibility and stability  | <p>Does the tool allow changing one concept to another?</p> <p>Will previously defined relationships be maintained when changing from one concept to another?</p> <p>Is the tool enough when editing or creating large and bulky models?</p>                     |
| Import/export functionality     | <p>Does the tool have import/export capabilities?</p> <p>Can the tool make information available to third parties (who do not work with the tool or use other tools)?</p> <p>Can (large) PDF files be published for use as posters and information exchange?</p> |
| Support                         | <p>Is the tool supported by a good help desk and user community?</p>                                                                                                                                                                                             |

Capturing architecture artifacts (catalogs, diagrams, maps, and matrices) in an architecture tool that provides the ability to publish them (to a website) helps increase architecture maturity. This is an important consideration when choosing a tool. It is an easy step to higher maturity.

The factors that ultimately determine the choice of an architecture tool largely depend on the situation. The same goes for the requirements for using a tool. These requirements are not easy to articulate – there is no standard set of requirements – because they are different for everyone.

The architecture tool selection described in this section should result in a product that can be used to create and maintain an Architecture Repository. At a minimum, the selected product should be able to provide a home for artifacts related to the organization's Architecture and Solutions Landscapes, Reference and Standards Libraries, and Architecture Requirements Repository. This is illustrated in Figure 7-22.



**Figure 7-22.** Minimum Architecture Repository components that must be represented in an architecture tool



In summary, a repository tool should offer functionality in order to support what is minimally needed to manage an Enterprise Architecture. Information that is not directly required for the Enterprise Architecture should not be included in the repository.

---

**A well-managed repository is characterized by ruthlessly minimizing information collected and maintained [18].**

---

In practice, however, a repository often does not work as described in the TOGAF Standard. There are few, if any, architecture tools that provide a one-size-fits-all solution that conforms to the framework's guidelines. For example, nine out of ten architecture tools do not distinguish between the Architecture Landscape and the Solutions Landscape. Of course, a good tool will allow its user to make this distinction based on a folder structure or something similar.

It's imperative to design the structure of the repository based on the architectural artifacts and the needs of the organization. This can be as simple or as complex as needed. For example, a hierarchical structure with folders and subfolders would be an excellent first step. It is important that the structure is intuitive and easy to navigate.

## 7.8. Summary

The Architecture Repository serves as a collection of architectures and solutions created during the architecture effort. When looking at the repository through a lens, a continuum can be used to visualize the state of the architecture work.

The TOGAF Standard recognizes three types of continua:

- Enterprise Continuum
- Architecture Continuum
- Solutions Continuum

The first of the three, the *Enterprise Architecture Continuum*, serves as an umbrella for the other two. The *Architecture Continuum* contains architectures that can be used as references to work toward an organization-specific architecture. There are four types of architectures in the Architecture Continuum: Foundation Architectures, Common Systems Architectures, Industry Architectures, and Organization-Specific Architectures. Each of these types of architectures contains an *increasing level of detail* as one moves through the Architecture Continuum from left to right.

The *Solutions Continuum*, the last of the three continua, contains four types of solutions. These are Foundation Solutions, Common Systems Solutions, Industry Solutions, and Organization-Specific Solutions. Again, moving from left to right provides an increasing level of detail about the solutions.

Catalogs, diagrams, maps, and matrices are used to capture solutions in the Architecture Repository. It is important to note that catalogs are *lists of building blocks* of a specific entity, type, or related types that are used for governance or reference purposes. A catalog addresses a specific entity and supplements the information about the concept with additional data. In a catalog, entities are not linked to other entities.

Diagrams and maps are *representations of architectural content* in either *graphical format* (diagrams) or *textual format* (maps). Diagrams and maps can also be used as a technique for graphically populating architectural content or for verifying the completeness of collected information. A diagram or map visualizes information captured in a catalog or matrix.

Lastly, matrices are a grid that represents the relationships between two or more model entities. The relationship between the entities – typically spread across two or more catalogs – is brought together in a matrix. The method of bringing entities together is called *cross-mapping*. Catalogs, diagrams, maps, and matrices are used to visualize building blocks. Architecture artifacts consist of one or more of these building blocks, and one or more artifacts make up an architecture deliverable.

When moving through the phases of the Architecture Development Method, a number of architecture artifacts may be produced. The number and composition of these artifacts largely depends on the level of customization of the TOGAF Standard. Once an organization has adapted the Enterprise Metamodel to its specific situation or requirements, an overview of the necessary architecture artifacts is formed based on this model. It is important to determine which artifacts add value to the architecture effort at hand and then create those artifacts. All other artifacts can then be ignored.

Building blocks – the components that make up an architecture artifact – are the backbone of the architecture and serve as essential, reusable components in architectural solutions. These blocks form a modular foundation, designed to be generic and adaptable, ensuring they can seamlessly integrate with other building blocks. Flexible as they are, building blocks can be defined at varying levels of detail, depending on the stage of architecture development. In the early stages, a building block might just have a name or a brief description. As development progresses, it can be broken down into supporting components, complete with detailed specifications. Building blocks serve as key references for both architecture and solution design. They are packages of functionality defined to meet the business needs across an organization. They correspond to the entities used and defined in the Enterprise Metamodel.

The process of building block definition takes place gradually as the Architecture Development Method is followed, mainly in phases A, B, C, and D.

Building blocks in the TOGAF Standard come in two flavors: Architecture Building Blocks and Solution Building Blocks. The former represents the *what* and *why*, providing high-level, technology-agnostic capabilities. The latter dives into the *how*, focusing on practical implementation using specific technologies and products.

When architecture work begins and the Architecture Development Methodology is applied during an initial iteration, Architecture Building Blocks are typically created during this round. These building blocks

capture architecture requirements such as business, data, application, and technology requirements. The information stored in the building blocks from this activity can be used to direct and guide the development of Solution Building Blocks in a later phase of architecture development.

In contrast to the high-level character of Architecture Building Blocks, Solution Building Blocks are actually much more detailed in nature. In fact, these building blocks describe the entities that make up the parent Architecture Building Block. Solution Building Blocks define what products and components will implement the required functionality; they determine the actual implementation of (parts of) the architecture. Solution Building Blocks fulfil business requirements and are product- or vendor-aware, something that Architecture Building Blocks are not. Both Architecture Building Blocks and Solution Building Blocks are typically stored in the Architecture Repository.

In order to communicate the architecture – consisting of the aforementioned building blocks – the TOGAF Standard uses *views* and *viewpoints*. These two concepts address the concerns of the stakeholders involved. The concepts are similar, yet fundamentally different. A view is a *representation of a system or architecture from the perspective of a specific stakeholder*. It is essentially a visualization or presentation of the architecture that addresses particular concerns or interests of one or more stakeholders. A viewpoint *defines the perspective and rules for creating a view*. It is essentially a template or a set of guidelines for constructing views to address specific concerns. Viewpoints outline how to structure views and which stakeholders they are aimed at.

Viewpoints are like blueprints for constructing views. Multiple views can be created using a single viewpoint, but each view is tailored to specific stakeholder concerns.

In summary, a view is what you *actually present* or show to stakeholders, while a viewpoint is the *set of rules or guidelines* that dictate how you create that view. Views and viewpoints are part of the Reference Library of the Architecture Repository.

Mentioned at the beginning of this chapter, the Architecture Repository is an intangible concept and *is not* necessarily equivalent to an architecture tool. However, many organizations use a tool under the assumption that it is. Vendors of architecture tools offer a variety of features in their products that come close to meeting the requirements of an Architecture Repository. The use of an architecture tool is therefore essential.

Deciding which tool is best for the organization depends on several factors. The factors that ultimately determine the choice of an architecture tool largely depend on the situation. The same goes for the requirements for using a tool. These requirements are not easy to articulate – there is no standard set of requirements – because they are different for everyone.